

WerkstattPilot

Aufgabe 5: Objektorientierte Implementation / Programmierung (OOP)

Java-Prototyp für die fachliche Klasse Auftrag



Projektbild / Profilbild: digitale Auftragsabwicklung für KFZ-Werkstätten

Angabe	Wert
Schule / Lehrgang	Juventus Schule HF Zürich / HF Informatik Applikationsentwicklung
Modul	Software Engineering 1
Dozent	Arif Chughtai
Projekt	WerkstattPilot
Arbeitsgruppe	Arbeitsgruppe 3 / WerkstattPilot-Team
Autoren	Livio Luna, Dominic Lolos
Version	1.2
Status	Aktualisierte Version mit neuestem Java-Code

0.1 Prüfungsabdeckung Aufgabe 5

#	Bewertungsgegenstand	Abdeckung
1	Persistence-Schicht	Kapitel 5
2	Business-Schicht	Kapitel 4
3	Presentation-Schicht	Kapitel 6
4	Unit-Tests Persistence	Kapitel 7
5	Unit-Tests Business	Kapitel 7
6	Endbenutzertest	Kapitel 8
7	Javadoc	Kapitel 9, HTML wird später erzeugt
8	Präsentation	Kapitel 10
9	Dozentenfeedback	Kapitel 11
10	Arbeitsprotokoll	Kapitel 12
11	Schreibweise	Gesamtdokument und Code

0.2 Dokumentstatus

Version	Datum	Änderung	Verantwortlich
1.0	04.06.2026	Erste Dokumentation Aufgabe 5 erstellt.	WerkstattPilot-Team
1.1	05.06.2026	An neuen einfachen Java-Code angepasst: nur Java, kein Maven, kein JUnit, Mock-DB.	WerkstattPilot-Team
1.2	06.06.2026	Überarbeitung Darstellung, Rechtschreibung,	WerkstattPilot-Team

Inhaltsverzeichnis

0.1 Prüfungsabdeckung Aufgabe 5.....	2
0.2 Dokumentstatus	2
Inhaltsverzeichnis	3
1 Ziel und Grundlage der Implementation	4
2 Umfang und Annahmen	4
2.1 Bewusste Abgrenzung	4
3 Projektstruktur und Packages	5
4 Implementation der Business-Schicht	5
4.1 Methoden der Schnittstelle AuftragService	6
4.2 Fachliche Regeln im Code	6
5 Implementation der Persistence-Schicht	6
6 Implementation der Presentation-Schicht	7
7 Unit-Tests	7
8 Endbenutzertest	7
9 Javadoc und Code-Dokumentation	8
9.1 Befehl zum Erzeugen der Javadoc-HTML-Seiten	8
10 Vorbereitung der Präsentation	8
11 Rückmeldungen des Dozenten	8
12 Arbeitsprotokoll.....	9
13 Quellen und Referenzen	9

1 Ziel und Grundlage der Implementation

Dieses Dokument beschreibt die Umsetzung von Aufgabe 5 für das Projekt WerkstattPilot. Grundlage ist das Designmodell aus Aufgabe 4. Der Prototyp ist bewusst klein gehalten und beschränkt sich auf den geforderten Umfang der ersten Iteration.

Die erste Iteration fokussiert auf Auftrag anlegen sowie Auftragsübersicht mit Statusänderung. Für Aufgabe 5 wird deshalb die fachliche Klasse Auftrag umgesetzt.

Grundlage	Bedeutung für Aufgabe 5
Aufgabe 2	Projektumgebung, Java-Projekt, Packages und Mock-DB-Entscheid.
Aufgabe 3	Auftrag als zentrale fachliche Klasse und Statusmodell.
Aufgabe 4	Designmodell mit Presentation-, Business- und Persistence-Schicht.
Aufgabe 5	Java-Prototyp, Tests, ConsoleClient, Javadoc und Präsentation.

2 Umfang und Annahmen

Punkt	Umsetzung
Gewählte fachliche Klasse	Auftrag
Attribut 1	auftragsNummer: int
Attribut 2	status: Auftragsstatus
Statuswerte	OFFEN, IN_ARBEIT, WARTET_AUF_TEILE, FERTIG, STORNIERT
Speicherung	In-memory Mock-DB mit ArrayList<Auftrag>
Testdaten	hart im Code, keine Benutzereingaben

2.1 Bewusste Abgrenzung

Nicht umgesetzt	Begründung
Maven / JUnit	Es wird bewusst nur Basic Java ohne externe Bibliotheken verwendet.
Echte Datenbank	Aufgabe 5 verlangt Mock-DB statt DB.
GUI	Presentation-Schicht ist ein ConsoleClient.
Kunde, Fahrzeug, Beanstandung als Klassen	In Aufgabe 5 werden nur zwei Attribute der Klasse Auftrag umgesetzt.

3 Projektstruktur und Packages

Das Projekt ist ein einfaches Java-Projekt. Die Struktur entspricht der 3-Schichtenarchitektur aus Aufgabe 2 und Aufgabe

Package	Verantwortung
presentation	Start der Anwendung, ConsoleClient, Konsolenausgabe.
business	Auftrag, Status, Service, Service-Implementierung und Factories.
persistence	Repository-Schnittstelle, Mock-Repository und Repository-Factory.
test/business	Einfache Java-Tests für die Business-Schicht.
test/persistence	Einfache Java-Tests für die Persistence-Schicht.

```
WerkstattPilot/  
  .project  
  .classpath  
  src/main/java/ch/werkstattteam/werkstattpilot/  
    presentation/  
    business/  
    persistence/  
  src/test/java/ch/werkstattteam/werkstattpilot/test/  
    business/  
    persistence/
```

4 Implementation der Business-Schicht

Die Business-Schicht enthält die fachliche Logik für Auftrag anlegen, Auftrag lesen, alle Aufträge lesen und Status ändern.

Klasse / Schnittstelle	Umsetzung im Code
Auftrag	Fachliche Klasse mit genau zwei Attributen: auftragsNummer und status.
Auftragsstatus	Enum mit den erlaubten Statuswerten.
AuftragFactory	Erstellt neue Auftrag-Objekte mit Status OFFEN.
AuftragService	Schnittstelle für die fachlichen Operationen.
AuftragServiceImpl	Konkrete Umsetzung der fachlichen Operationen.
AuftragServiceFactory	Einfache Factory als Singleton für den Service.

4.1 Methoden der Schnittstelle AuftragService

Methode	Rückgabe	Parameter	Zweck
auftragAnlegen	Auftrag	int auftragsNummer	Erstellt und speichert einen neuen Auftrag.
auftragLesen	Auftrag	int auftragsNummer	Liest einen Auftrag anhand der Nummer.
alleAuftraegeLesen	List<Auftrag>	-	Liest alle gespeicherten Aufträge.
statusAendern	Auftrag	int auftragsNummer, Auftragsstatus neuerStatus	Ändert den Status eines bestehenden Auftrags.

4.2 Fachliche Regeln im Code

Regel	Umsetzung
Neuer Auftrag	Ein neuer Auftrag wird mit Status OFFEN erstellt.
Auftragsnummer	Die Auftragsnummer muss grösser als 0 sein.
Status	Ein leerer Status wird beim Erstellen als OFFEN gesetzt; beim Ändern darf Status nicht leer sein.
Unbekannter Auftrag	Beim Statuswechsel eines unbekannten Auftrags wird eine IllegalArgumentException geworfen.

5 Implementation der Persistence-Schicht

Die Persistence-Schicht speichert und liest Auftrag-Objekte. Es wird keine echte Datenbank verwendet.

AuftragRepositoryMock speichert die Daten während der Programmausführung in einer ArrayList im Hauptspeicher.

Klasse / Schnittstelle	Umsetzung im Code
AuftragRepository	Interface für Speichern, Lesen, alle Lesen und Aktualisieren.
AuftragRepositoryMock	Mock-DB mit ArrayList<Auftrag>.
AuftragRepositoryFactory	Einfache Factory als Singleton für das Repository.
Methode	Beschreibung
speichern(Auftrag auftrag)	Speichert einen neuen Auftrag in der Liste. Doppelte Nummern sind nicht erlaubt.
lesen(int auftragsNummer)	Sucht einen Auftrag anhand der Auftragsnummer.
alleLesen()	Gibt eine Kopie der Liste zurück.
aktualisieren(Auftrag auftrag)	Sucht einen Auftrag in der Liste und ersetzt ihn.

6 Implementation der Presentation-Schicht

Die Presentation-Schicht besteht aus WerkstattPilotApp und ConsoleClient. Die Klasse WerkstattPilotApp enthält die main-Methode. Der ConsoleClient führt den Endbenutzertest aus. Es gibt keine Eingaben über die Tastatur.

Klasse	Verantwortung
WerkstattPilotApp	Startklasse mit main-Methode. Erstellt ein ConsoleClient-Objekt und startet dieses.
ConsoleClient	Erstellt Testdaten, ruft den AuftragService auf und gibt die Aufträge auf der Konsole aus.

7 Unit-Tests

Die Tests sind einfache Java-Testklassen ohne externe Testbibliothek. Jede Testklasse besitzt eine main-Methode und eine eigene pruefe-Methode. Wenn eine Prüfung fehlschlägt, wird eine RuntimeException geworfen.

Testklasse	Testfälle / Prüfung
AuftragRepositoryTest	speichernUndLesenTest, alleLesenTest, aktualisierenTest
AuftragServiceTest	auftragAnlegenTest, statusAendernTest, alleAuftraegeLesenTest, tatusAendernMitUnbekannterNummerTest

8 Endbenutzertest

Der Endbenutzertest wird über WerkstattPilotApp gestartet. Der ConsoleClient erstellt drei Aufträge und ändert anschliessend Statuswerte.

Auftragsnummer	Startstatus	Status nach Ablauf
1001	OFFEN	FERTIG
1002	OFFEN	IN_ARBEIT
1003	OFFEN	WARTET_AUF_TEILE

WerkstattPilot - Endbenutzertest Aufgabe 5

Erfasste Auftraege:

Auftrag 1001 - Status: OFFEN

Auftrag 1002 - Status: IN_ARBEIT

Auftrag 1003 - Status: WARTET_AUF_TEILE

Status von Auftrag 1001 wird auf FERTIG gesetzt.

Auftraege nach Statusaenderung:

Auftrag 1001 - Status: FERTIG

Auftrag 1002 - Status: IN_ARBEIT

Auftrag 1003 - Status: WARTET_AUF_TEILE

9 Javadoc und Code-Dokumentation

Die Java-Dateien enthalten Dateiköpfe und einfache Javadoc-Kommentare. Dokumentiert sind Klassen, Schnittstellen, Factories, wichtige Methoden sowie Getter und Setter. Die HTML-Seiten werden nach dem finalen Code-Stand erzeugt.

Vorgabe	Umsetzung im Code
Dateiköpfe	Jede Java-Datei enthält Projekt, Aufgabe, Datei und Autoren.
Interfaces	AuftragService und AuftragRepository sind dokumentiert.
Factories	AuftragFactory, AuftragServiceFactory und AuftragRepositoryFactory sind dokumentiert.
Getter / Setter	Getter und Setter in Auftrag sind dokumentiert.

9.1 Befehl zum Erzeugen der Javadoc-HTML-Seiten

```
javadoc -encoding UTF-8 -charset UTF-8 -docencoding UTF-8 -d javadoc_html -sourcepath
src/main/java -subpackages ch.werkstattteam.werkstattpilot
```

Nach der Ausführung liegt die Startseite unter javadoc_html/index.html. Der Ordner javadoc_html wird anschliessend der Abgabe beigelegt.

10 Vorbereitung der Präsentation

Teil	Inhalt
Projektumgebung	Java-Projekt, 3-Schichtenstruktur und Mock-DB erklären.
Analyse / Design	Auftrag als zentrale Klasse und Repository/Factory kurz erklären.
Prototyp	WerkstattPilotApp starten und Konsolenausgabe zeigen.
Tests / Javadoc	Testklassen starten und danach Javadoc-HTML zeigen.

11 Rückmeldungen des Dozenten

Datum	Rückmeldung	Umsetzung / Einarbeitung	Status
Di, 21.04.2026	Wir sind auf dem guten Weg.	Projektumfang nicht unnötig erweitert.	berücksichtigt
Di, 05.05.2026	Im Grossen und Ganzen gut. Einzelne Konsistenzen noch offen.	MariaDB ausgebaut; nur Java und Mock-DB umgesetzt.	umgesetzt
05.06.2026	Kein neues Feedback zu Aufgabe 5 vorhanden.	Dokument und Code aufeinander abgestimmt.	offen für Ergänzung

12 Arbeitsprotokoll

Datum	Beteiligte	Tätigkeit	Ergebnis	Status
05.06.2026	Livio Luna	Umfang festgelegt	Auftrag und zwei Attribute gewählt.	erledigt
05.06.2026	Livio Luna	Java-Projekt vereinfacht	Kein Maven, kein JUnit, keine echte DB.	erledigt
05.06.2026	Livio Luna	Business und Persistence umgesetzt	Service, Repository, Mock-DB und Factories erstellt.	erledigt
05.06.2026	Livio Luna	Tests ausgeführt	Business- und Persistence-Tests erfolgreich.	erledigt
06.06.2026	Arbeitsgruppe	Javadoc HTML erzeugen	javadoc_html wird noch erstellt.	erledigt

13 Quellen und Referenzen

Quelle	Verwendung im Projekt
Aufgabenblatt Aufgabe 5	Vorgaben für Java-Prototyp, Tests, ConsoleClient und Javadoc.
Aufgabe 2 - Projektumgebung	Grundlage für Java, Packages und Mock-DB.
Aufgabe 3 - Analysemodell	Grundlage für Auftrag und Status.
Aufgabe 4 - Designmodell	Grundlage für 3-Schichtenstruktur, Service, Repository, Factory.
Java-Dokumentation / JDK	Grundlage für Java-Code und Javadoc-Erzeugung.